

Sorting Algorithms – Performance Analysis

SPA Assignment Report | April 2026

1. Timing Results

All times are in milliseconds (ms). Experiments used random seed 42, integers in [1, 100 000]. Each algorithm sorted a fresh copy of the data.

Input Type	Size	Insertion Sort (ms)	Merge Sort (ms)	Quick Sort (ms)
Random	1000	13.4720	1.3431	0.7959
Random	5000	379.1722	9.2852	5.4671
Random	10000	1522.8583	16.6486	11.0892
Sorted	1000	0.0645	0.9795	44.9301
Sorted	5000	0.4437	5.7715	921.3740
Sorted	10000	0.8526	19.5595	3514.4049
Reverse	1000	24.4621	1.6539	24.7182
Reverse	5000	772.3283	5.8139	648.8902
Reverse	10000	2950.7426	11.5268	2298.0027

2. Comparison by Input Type

2.1 Random Input

On random data, Quick Sort was the fastest across all sizes, followed closely by Merge Sort. Both ran in $O(n \log n)$ time, so their times grew roughly proportionally to $n \times \log(n)$. Insertion Sort was dramatically slower — at $n = 10\,000$ it took $\sim 1\,523$ ms versus ~ 11 ms for Quick Sort — demonstrating the $O(n^2)$ penalty in practice.

2.2 Already-Sorted Input

Insertion Sort became extremely fast on sorted data (0.85 ms at $n = 10\,000$) because each element is already in place: the inner loop executes zero comparisons per element, giving $O(n)$ behaviour. Merge Sort maintained its steady $O(n \log n)$ regardless. Quick Sort with a last-element pivot suffered its worst case here: the pivot is always the largest element, creating maximally unbalanced partitions and pushing time to $\sim 3\,514$ ms — over $300\times$ slower than Merge Sort.

2.3 Reverse-Sorted Input

Insertion Sort was slowest ($\sim 2\,951$ ms) because every element must be shifted all the way to the front, maximising the $O(n^2)$ work. Merge Sort again performed best (~ 11.5 ms) with its input-independent complexity. Quick Sort was also slow ($\sim 2\,298$ ms) because the last-element pivot on a descending list

repeatedly selects the minimum value, again yielding $O(n^2)$ partitioning depth.

3. Theory vs. Measured Times

The theoretical complexities align well with observations:

Algorithm	Best Case	Average Case	Worst Case	Space
Insertion Sort	$O(n)$	$O(n^2)$	$O(n^2)$	$O(1)$
Merge Sort	$O(n \log n)$	$O(n \log n)$	$O(n \log n)$	$O(n)$
Quick Sort	$O(n \log n)$	$O(n \log n)$	$O(n^2)$	$O(\log n)$

When sizes grew from 1 000 to 10 000 (10×), Merge Sort times grew by roughly 10–12×, consistent with $O(n \log n)$. Insertion Sort times grew ~100–120× on random and reverse inputs, matching $O(n^2)$. These ratios confirm that the asymptotic models predict real behaviour reliably once n is large enough.

4. Quick Sort – Worst-Case Behaviour

With the Lomuto partition scheme and a last-element pivot, Quick Sort degrades to $O(n^2)$ whenever the pivot is consistently the minimum or maximum element. This happens on:

- **Sorted input:** the last element is always the maximum, producing partitions of size $(n-1)$ and 0 at every level — n recursive calls, each doing $O(n)$ work. Observed: 3 514 ms at $n = 10\,000$.
- **Reverse-sorted input:** the last element is always the minimum, mirroring the same imbalance. Observed: 2 298 ms at $n = 10\,000$.

Common mitigations include choosing a random pivot, or using the median-of-three strategy, which restore the expected $O(n \log n)$ performance on these adversarial inputs.

5. Stability and Memory Properties

Property	Insertion Sort	Merge Sort	Quick Sort
Stable?	Yes	Yes	No (Lomuto)
In-place?	Yes – $O(1)$	No – $O(n)$ aux	Yes – $O(\log n)$ stack
Comparisons	$O(n^2)$ avg	$O(n \log n)$	$O(n \log n)$ avg
Best for	Small / nearly sorted	Guaranteed $O(n \log n)$ needed	General-purpose large data

Insertion Sort is stable because equal elements are never swapped past each other (the strict $>$ comparison ensures this). It is in-place, needing only a single temporary variable. **Merge Sort** is stable because the merge step always picks the left element first when values are equal. It requires $O(n)$ auxiliary memory for the temporary arrays during merging. **Quick Sort** (Lomuto) is generally not stable; swaps during partitioning can re-order equal elements. It is effectively in-place — only $O(\log n)$ stack space for recursion on average.

Environment: Python 3.12 · Seed 42 · Integers $\in [1, 100\,000]$